SENIOR .NET BACKEND & ARCHITECTURE INTERVIEW PACK

Large-scale data integration • REST APIs • Messaging • Reliability • Cloud

Role context

Platform connects tens of thousands of veterinary clinics into a central analytics/data platform.

The interview is likely to test production integration ownership, safe async processing, correctness at scale, and judgment in an existing mature codebase.

How to use this pack

Answer each question out loud before reading the model answer.

For senior-level answers, state the trade-off, failure mode, and how you would observe it in production.

Do not memorize wording. Practice explaining the reasoning and adapting it to the constraints the interviewer gives you.

Prepared as an architect-led interview simulation based on the supplied job description.

1. What I would test as the lead architect

- Can you design integrations that survive retries, duplicate delivery, rate limits, partial failures, and schema variation?
- Can you reason about data correctness, not only throughput?
- Can you choose between synchronous APIs, queues, streams, scheduled jobs, and database coordination?
- Can you diagnose production failures using logs, metrics, traces, correlation IDs, and replayable audit data?
- Can you safely change a mature system you did not design?
- Can you defend a technical decision with evidence while still being pragmatic with clients and senior engineers?

Likely senior interview shape

15-20 min: experience and integration ownership

30-45 min: system design / data-import scenario

20-30 min: .NET, APIs, SQL, messaging and testing depth

10-15 min: production incident / legacy-code / client-pushback scenarios

2. Architecture & system design

1. Design the ingestion architecture for tens of thousands of clinics. Some support APIs, some databases/files, some webhooks. We need historical imports plus continuous sync.

Strong answer

I would separate source-specific extraction from the canonical ingestion pipeline. Each connector owns authentication, pagination, throttling and source quirks, but emits a normalized ingestion envelope containing source system, tenant/clinic, entity type, source record ID, source version or timestamp, correlation/import ID, payload and schema version.

- Historical backfills become partitioned import jobs with explicit checkpoints. Continuous sync uses webhooks or incremental polling/CDC when available.
- A durable queue or stream decouples extraction from transformation/import so downstream slowdowns do not overload clinics.
- Transformation and validation are versioned and deterministic. Bad records go to a quarantine/dead-letter workflow rather than blocking the whole clinic.
- Writes into the central model are idempotent using a stable source key plus version semantics. Every stage records status and correlation metadata for traceability.
- Autoscaling is driven by backlog/partition load, but concurrency is bounded per clinic and per external API to respect rate limits and avoid noisy-neighbor problems.

I would optimize for recoverability and correctness first, then throughput. At this scale, retrying safely is more valuable than pretending failures will be rare.

Architect follow-up probes

- What is the unit of partitioning: clinic, entity, import batch, or something else?
- How do you prevent a single huge clinic from monopolizing workers?
- Where do you persist checkpoints?
- How do you handle records that reference other records not imported yet?

What would concern me

- One giant ETL job with in-memory progress only.
- Saying “Kafka solves scale” without defining keys, ordering, replay, and consumer behavior.
- No strategy for partial failure or source throttling.

2. How do you make a multi-million-record historical import resumable?

Strong answer

The import needs a durable job model. I would persist job state and checkpoint at a granularity that makes retries inexpensive: page/cursor, source primary-key range, file chunk, or deterministic partition.

- Each work item has a stable ID and states such as Pending, Running, Completed, Failed/Retryable, Quarantined.
- Checkpoint advancement happens only after the target write and required side effects are durably committed.
- Workers use leases or visibility timeouts so abandoned work becomes available again.
- The import operation itself is idempotent, so replaying the last completed boundary is safe.
- Progress is derived from durable state, not only logs.

For a database source, keyset pagination by a stable indexed key is usually safer than OFFSET at large volumes. For APIs, I preserve the provider cursor where its semantics are reliable.

Architect follow-up probes

- Would you checkpoint every record? Why or why not?
- What if the source data changes while the historical import is running?
- How would you pause and resume a clinic intentionally?

What would concern me

- Restarting the complete import after any failure.
- Checkpoint updated before target persistence.

3. Explain idempotency in this platform. Where would you implement it?

Strong answer

Idempotency means processing the same logical operation more than once produces the same externally visible result as processing it once. It is essential because distributed systems commonly deliver at least once.

- At API boundaries, accept or derive an idempotency key and persist the result/status for that operation.
- For imported entities, use a durable source identity such as (clinicId, sourceSystem, entityType, sourceRecordId), often with source version/updated-at semantics.
- For messages, keep a consumer inbox/deduplication record or make the database mutation naturally idempotent through constraints/upserts/version checks.
- For outbound effects, use an outbox so committing business state and publishing intent happen atomically in one database transaction.

Idempotency is not “ignore duplicate HTTP requests in memory”; the deduplication decision has to survive process restarts and concurrent workers.

Architect follow-up probes

- How long do you retain idempotency keys?
- What if the same key arrives with a different payload?
- Is UPSERT alone always sufficient?

What would concern me

- Using only an in-memory cache.
- Claiming exactly-once delivery removes the need for idempotency.

4. How would you handle ordering when records have dependencies, for example client -> patient -> visit -> invoice?

Strong answer

I would avoid requiring global ordering. I would identify the smallest key that needs ordering, such as clinic plus aggregate/entity ID, and partition messages by that key where possible.

- For historical imports, I may stage raw source data first and resolve dependencies in deterministic phases.
- For streaming updates, I include source versions or sequence numbers and reject/park stale or impossible transitions.
- If a child arrives before its parent, options include delayed retry, a pending-reference table, or creating a controlled placeholder only if the domain permits it.

The important distinction is transport ordering versus business consistency. Even an ordered partition can be replayed or have poison messages, so handlers still need version and idempotency rules.

Architect follow-up probes

- Would you use one Kafka partition per clinic?
- What happens when an older update arrives after a newer update?

5. Would you choose Kafka, Azure Service Bus/RabbitMQ, or both?

Strong answer

I choose based on semantics rather than popularity. A broker such as Azure Service Bus or RabbitMQ is excellent for commands/work queues: competing consumers, retries, dead-lettering, scheduling and explicit job processing. Kafka is strong when I need an append-only event log, high-throughput ordered partitions, multiple independent consumers, retention and replay.

- For import work orchestration, a queue is often simpler.
- For normalized domain/data-change events that several analytics consumers need to replay independently, Kafka can be a strong fit.
- Using both is reasonable only if the platform genuinely needs both interaction models; otherwise it increases operational complexity.

I would ask about existing cloud services, throughput, replay requirements, ordering scope, team operational experience and retention before selecting.

Architect follow-up probes

- How do consumer groups work in Kafka?
- What is a dead-letter queue good for, and what is it not good for?
- How do you avoid infinite poison-message retries?

6. How do you design for backpressure and rate limits across thousands of third-party systems?

Strong answer

I would put a durable buffer between acquisition and processing, then apply concurrency controls at several levels. A global worker limit protects our platform; per-provider and per-clinic limits protect external systems; token-bucket/leaky-bucket style rate limiters enforce known quotas.

- Honor Retry-After and distinguish 429 from transient 5xx and permanent 4xx failures.
- Use exponential backoff with jitter and cap retries.
- Track queue age, throttling time, retry rates and provider-specific success latency.
- For huge backfills, schedule fairly across clinics rather than greedily draining one source.

A circuit breaker can reduce pressure during sustained provider failure, but it does not replace correct retry classification or rate limiting.

Architect follow-up probes

- Why jitter?

- Should 401 be retried?
- How do you prevent a retry storm after an outage?

3. REST APIs, OAuth2, webhooks & integration ownership

1. Walk me through how you would build a production-grade third-party REST integration.

Strong answer

I treat an integration as a product boundary, not an HttpClient call. I define authentication/token lifecycle, timeout budget, retry classification, rate limiting, pagination/cursors, schema mapping, validation, idempotency, observability, test doubles/contract tests and an operational runbook.

- Use IHttpClientFactory or a typed client so connection pooling and handler lifetime are managed correctly.
- Set explicit timeouts and pass CancellationToken through the call chain.
- Do not blindly retry non-idempotent operations unless the provider supports idempotency keys or we can safely determine the outcome.
- Persist external IDs and enough request/response metadata to reconcile failures, while redacting secrets and sensitive data.
- Version the mapping layer so provider schema changes are isolated from core domain code.

Architect follow-up probes

- What do you log from a failed provider request?
- How would you detect a silent schema change?

2. Explain OAuth2 client credentials and token handling for server-to-server integrations.

Strong answer

With client credentials, the service authenticates itself to the authorization server and obtains an access token for configured scopes. There is usually no refresh token; when the access token is near expiry, the service requests another one using its client credentials.

- Cache tokens by audience/scope and refresh slightly before expiry to avoid a thundering herd at the boundary.
- Store client secrets/certificates in a secret manager, never configuration committed to source control.
- On 401, first distinguish expired/invalid token from authorization/configuration problems; one controlled token refresh retry can be reasonable, not an unbounded retry loop.
- Prefer certificate/client assertion or managed identity where the provider supports it.

Architect follow-up probes

- Difference between authentication and authorization?
- What changes for authorization-code flow?

3. How do you implement reliable webhooks?

Strong answer

I assume webhooks can be duplicated, delayed, arrive out of order and occasionally never arrive. The endpoint authenticates/verifies the sender, validates minimal envelope data and returns quickly after durably recording or enqueueing the event.

- Verify HMAC/signature over the exact raw body according to the provider specification, including timestamp/replay protection when offered.
- Deduplicate using provider event ID or a stable computed key.
- Do heavy processing asynchronously.
- Persist event status and correlation information so support can replay or inspect it.

- Use periodic reconciliation/polling for business-critical integrations because webhooks alone should not be the only consistency mechanism unless the provider guarantees that contract.

Architect follow-up probes

- When do you return 2xx versus 4xx/5xx?
- What if the signature is valid but mapping fails?

4. How would you expose an API endpoint that starts a long-running import?

Strong answer

I would not keep the HTTP request open for a large import. The API validates the request, creates a durable import job, returns 202 Accepted with a job ID and status URL, and queues work asynchronously.

- GET /imports/{id} exposes state, progress, timestamps and summary errors.
- A client-supplied idempotency key prevents duplicate job creation.
- Cancellation/pause can be modeled explicitly if the business needs it.
- The job API should not claim success merely because work was accepted; accepted, running, partially completed, failed and completed are distinct states.

Architect follow-up probes

- Would you use SignalR for progress?
- How would you secure status visibility per tenant?

4. Modern .NET / ASP.NET Core depth

1. What are common HttpClient mistakes in .NET and how do you avoid them?

Strong answer

Creating and disposing a new HttpClient for every request can cause connection churn/socket exhaustion. Keeping one forever can mishandle DNS changes depending on handler configuration. IHttpClientFactory/typed clients provide managed handler pooling and a clean place for policies/configuration.

- Set timeout budgets intentionally; avoid stacking multiple retry layers.
- Use HttpRequestMessage for per-request headers rather than mutating shared DefaultRequestHeaders concurrently.
- Stream large payloads instead of buffering everything when possible.

Architect follow-up probes

- What does PooledConnectionLifetime solve?

2. Explain async/await problems you look for in backend code.

Strong answer

I look for sync-over-async (.Result/.Wait), unbounded parallelism, forgotten CancellationToken propagation, fire-and-forget tasks with no ownership, async work hidden inside locks, and excessive Task.Run around naturally asynchronous I/O.

- For I/O-bound work, async improves scalability by not occupying a thread while waiting.
- For controlled parallel calls, I use bounded concurrency such as SemaphoreSlim, Channels or worker queues rather than Task.WhenAll over tens of thousands of items.
- Background work belongs in a hosted service or external worker with durable state if it must survive process restarts.

Architect follow-up probes

- Why can Task.WhenAll still be dangerous?
- When is Task.Run appropriate on the server?

3. How would you structure validation and error handling in ASP.NET Core?

Strong answer

I separate transport validation from domain/business validation. Invalid request shape or simple constraints can fail at the API boundary. Domain rules live in application/domain logic so they are enforced regardless of entry point.

- A global exception handler maps known exceptions to consistent ProblemDetails responses and correlation IDs.
- Expected failures are not logged as fatal exceptions everywhere.
- Do not leak stack traces, tokens, connection strings or provider secrets to clients.

Architect follow-up probes

- 409 vs 422 vs 400 - when would you use each?

4. What lifetime would you choose for DbContext and why?

Strong answer

In a typical web request, DbContext is scoped. It represents a unit of work and is not thread-safe, so I do not share one DbContext across concurrent tasks.

- For background workers I create scopes per message/batch or use IDbContextFactory when appropriate.
- I keep transactions intentionally bounded; a huge import should not hold one database transaction for millions of records.

Architect follow-up probes

- What happens if multiple tasks use the same DbContext?

5. SQL, data correctness & large-volume processing

1. How do you load millions of records efficiently without sacrificing correctness?

Strong answer

I avoid one-row-at-a-time network round trips. Depending on the database and ORM constraints I use batching, bulk copy/COPY, staging tables, set-based merge/upsert patterns and appropriate indexes.

- Stage raw/normalized data first when validation/reconciliation is complex.
- Keep batch sizes bounded so transactions, locks and memory remain manageable.
- Use unique constraints to enforce identities that must never duplicate.
- Measure database write latency, lock waits, index maintenance cost and transaction-log/WAL pressure instead of optimizing only application CPU.

Architect follow-up probes

- What indexes would you need for idempotent upserts?
- Why can too many indexes hurt bulk import?

2. OFFSET pagination or keyset pagination?

Strong answer

For large changing datasets, keyset pagination is usually more stable and efficient: WHERE Id > @lastId ORDER BY Id LIMIT @n, ideally using a stable indexed ordering key. OFFSET becomes more expensive as the offset grows and can skip/duplicate rows when data changes.

- If the ordering key is not unique, use a composite cursor such as (UpdatedAt, Id).
- The source semantics matter: snapshot isolation or a high-water mark may be needed for a consistent historical extract.

Architect follow-up probes

- What if IDs are GUIDs?
- How do you resume with UpdatedAt ties?

3. How do you prevent lost updates when multiple workers process the same logical record?

Strong answer

I start by preventing unnecessary concurrency through partitioning, but the database still needs a correctness guard. Options include optimistic concurrency with a version/rowversion, compare-and-swap on source version, unique constraints plus upsert, or explicit locking for narrow critical sections.

- Last-write-wins is only valid if that is the domain rule, not as a default technical shortcut.
- A source version/sequence lets us reject stale messages deterministically.

Architect follow-up probes

- When would pessimistic locking be justified?

4. How do you investigate a suddenly slow SQL-backed import?

Strong answer

I correlate application latency with database metrics: execution plans/query store, waits, locks/blocking, CPU/IO, temp usage, log/WAL pressure, connection-pool saturation and row counts/cardinality changes.

- I compare the slow query plan with a known good period if possible.
- I check whether a deployment changed SQL shape, indexes, batch size or parallelism.
- I avoid “add an index” until I know which access pattern and bottleneck I am fixing.

Architect follow-up probes

- What causes parameter-sniffing/plan instability?

6. Messaging, event-driven architecture & delivery guarantees

1. Explain at-most-once, at-least-once and exactly-once in practical terms.

Strong answer

At-most-once can lose a message but avoids redelivery. At-least-once retries until acknowledged, so duplicates are possible and consumers must be idempotent. “Exactly once” is usually scoped to a particular broker/transactional boundary; it does not magically make external APIs and databases exactly-once together.

- For business integrations I usually design around at-least-once delivery plus idempotent handlers because it is robust and understandable.

Architect follow-up probes

- Can Kafka exactly-once semantics prevent duplicate emails or third-party POSTs?

2. Explain the outbox pattern and why it matters.

Strong answer

The outbox solves the dual-write problem. In the same database transaction as the business state change, I insert an outbox record describing the event/message. A separate publisher sends unsent outbox rows to the broker and marks them published.

- If publishing is retried, downstream consumers still need idempotency.
- I monitor outbox age/backlog because a stuck publisher is a production incident even though the business transaction succeeded.

Architect follow-up probes

- How do you handle multiple publisher instances?

3. Explain an inbox pattern for consumers.

Strong answer

Before applying a message, the consumer records the message ID in an inbox/deduplication table within the same transaction as the business mutation. If the unique message ID already exists, it is a duplicate and the handler can acknowledge without reapplying the effect.

- Retention can be bounded according to broker redelivery/replay semantics and business needs.
- The message ID must represent the logical event, not a new random ID generated on every retry.

Architect follow-up probes

- Could a natural business key replace an inbox table?

4. What is your dead-letter strategy?

Strong answer

A DLQ is an operational isolation mechanism, not a trash can. I dead-letter after classifying a message as non-retryable or after bounded retries are exhausted.

- Capture reason, original message metadata, exception category, attempts and correlation ID.
- Alert on DLQ growth/rate, not just absolute count.
- Provide a controlled replay tool after the underlying issue or data is corrected.
- Do not replay blindly into the same deterministic failure loop.

Architect follow-up probes

- Which failures should go immediately to DLQ?

7. Reliability, observability & production support

1. What would you monitor for this platform?

Strong answer

I would cover user/business health, pipeline health and infrastructure health.

- Business/pipeline: records imported, success/error/quarantine counts, per-clinic freshness/lag, reconciliation mismatch, import completion time.
- Messaging: queue depth, oldest-message age, retry rate, DLQ count/rate, consumer throughput.
- Integrations: provider latency, 2xx/4xx/5xx/429 rates, token failures, webhook lag.
- Database: query latency, connection pool, blocking/deadlocks, CPU/IO, storage/log pressure.
- Service: request rate, error rate, p95/p99 latency, saturation, GC/memory where relevant.

I prefer alerts tied to symptoms/SLOs - for example clinic freshness falling behind - rather than only machine metrics.

Architect follow-up probes

- What is a good SLI for import health?
- Would you alert on every exception?

2. Logs, metrics and traces - what does each give you?

Strong answer

Logs explain discrete events with context, metrics show aggregate behavior over time, and distributed traces show the causal path/latency across service boundaries. I use all three with a common correlation/import/trace identity.

- Structured logs use named fields such as ClinicId, ImportId, Provider, EntityType, MessageId and attempt number.
- OpenTelemetry gives vendor-neutral instrumentation for traces/metrics/log correlation, then data can flow to Application Insights, Datadog, Grafana-compatible backends, etc.

Architect follow-up probes

- What should never be logged?

3. Production import is failing for 2% of clinics after a deployment. What do you do first?

Strong answer

I first contain impact and establish scope: which clinics/providers/entity types, when it started, whether failure rate is growing, and whether data correctness is at risk. I correlate with the deployment and inspect representative traces/logs and DLQ/quarantine reasons.

- If rollback is safe and the deployment is strongly correlated, rollback/feature-disable can be the fastest containment.
- I preserve failed work for replay; I do not delete queues or restart everything blindly.
- I separate transient external failure from deterministic mapping/data regression.
- After recovery I reconcile affected clinics and document the prevention: test, guardrail, metric or deployment check.

Architect follow-up probes

- What evidence would justify rollback?
- How do you prove no records were silently lost?

8. Legacy system & safe modernization

1. You join a large codebase with poor documentation. How do you get to a trustworthy mental model quickly?

Strong answer

I start with behavior, not assumptions. I trace one or two real business flows end to end: entry point, application logic, database writes, queues/external calls, background processors and operational dashboards.

- I use logs/traces, tests, schema and deployment topology to validate what the code appears to do.
- I identify ownership boundaries, high-risk modules, integration contracts and known incidents.
- Before significant change, I add characterization/contract tests around the behavior we cannot break.
- I document the real dependency/data flow as I learn it and validate it with existing owners.

Architect follow-up probes

- What if existing tests conflict with production behavior?

2. How do you improve a fragile integration without a rewrite?

Strong answer

I introduce seams and observability first. Wrap the external provider behind an explicit adapter/client contract, centralize timeout/retry/auth behavior, add tests around current mapping, and measure failure modes.

- Then I replace behavior incrementally behind feature flags or by clinic/provider cohorts.
- Use shadow comparison or dual-read where feasible to compare old/new outputs without immediately changing the source of truth.
- Define rollback/replay before rollout.

A rewrite is justified only when the migration and operational risk is lower than continued incremental change - that is uncommon without evidence.

Architect follow-up probes

- How would you prove the new mapper is equivalent?

3. When would you refactor versus leave ugly code alone?

Strong answer

I refactor when the current structure materially increases change risk, defect rate, operational cost or blocks a required feature. I avoid aesthetic rewrites in stable code with no business leverage.

- I prefer small refactors adjacent to a change, protected by tests and measurable behavior.
- If a dangerous area cannot be safely refactored now, I at least make the risk visible with tests, metrics, comments/ADRs and a deliberate follow-up.

Architect follow-up probes

- How do you sell that decision to a client who only wants the feature?

9. Automated testing strategy

1. What tests would you build for a third-party integration?

Strong answer

I use several layers rather than mocking everything.

- Unit tests for deterministic mapping, validation, retry classification and domain rules.
- HTTP integration/component tests using a controlled fake server to verify actual serialization, headers, pagination, OAuth/token refresh, timeouts and error responses.
- Database integration tests for idempotency, unique constraints, concurrency and transactional outbox/inbox behavior.
- Contract tests against provider sandbox/schema examples when available.
- A small number of end-to-end tests through queue + persistence for critical paths.
- Replay tests using sanitized real failure payloads are very valuable for integration-heavy systems.

Architect follow-up probes

- Why is mocking HttpClient interfaces often insufficient?

2. How do you test retry logic without making tests slow and flaky?

Strong answer

I separate retry policy from real time where possible and inject/fake delay or use a policy library/test hooks. The fake downstream returns a deterministic sequence such as 500, 500, 200 and I assert attempt count and final behavior.

- I test which status codes/exceptions are retryable, Retry-After handling, max attempts and cancellation.
- I avoid sleeping for seconds in tests.

Architect follow-up probes

- Should validation errors be retried?

10. Senior judgment & client-facing scenarios

1. A US client says: “Just add retries to everything; we need this fixed today.” How do you respond?

Strong answer

I would align on the urgency but separate failure categories. Blind retries can duplicate writes, amplify an outage and make rate limiting worse. I would show evidence from the incident - status codes, operation type and provider behavior - then propose the smallest safe fix.

- For a transient idempotent GET, bounded retry with backoff/jitter may be appropriate immediately.
- For a POST that can create duplicates, I first need provider idempotency support, a request key, or reconciliation logic.
- I would communicate impact, risk and rollback plan in business terms, not use architecture vocabulary as a veto.

Architect follow-up probes

- What if they still insist?

2. A senior engineer wants Kafka. You think Azure Service Bus is enough. How do you hold your position?

Strong answer

I would turn the debate into requirements: required throughput, ordering scope, retention/replay, number of independent consumers, operational capability, latency, cloud constraints and cost. Then compare both against those needs.

- If the only requirement is durable work distribution with retries/DLQ and no event-log/replay need, a queue may be simpler.
- If the platform needs retained event history and several independent consumers replaying ordered partitions, Kafka may justify its complexity.
- I am comfortable changing my recommendation if new evidence changes the constraints; holding a position means defending reasoning, not defending ego.

Architect follow-up probes

- How would you document the decision?

3. Tell me about a time you disagreed with a technical decision.

Strong answer

A strong interview answer should use a compact STAR structure but emphasize evidence and outcome: context, concrete technical risk, data or prototype used to validate the concern, how you communicated without blocking the team, the decision taken, and what happened in production.

- Good examples for this role: unsafe retries, a migration without rollback, overloading a third-party API, missing idempotency, schema change risk, or changing a legacy hot path without characterization tests.

- Avoid framing the story as “I was the smartest person in the room.” The senior signal is that the decision became better and the team retained trust.

Architect follow-up probes

- What would you do differently now?

4. How do you balance delivery speed and engineering quality?

Strong answer

I make the trade-off explicit. I distinguish reversible shortcuts from irreversible data/correctness risk. For business-critical integration code, I will compress scope before I compromise idempotency, security, rollback or observability.

- A temporary solution should have an owner, bounded blast radius, instrumentation and a removal condition/date if the debt is meaningful.
- I communicate consequences in terms of delivery risk: “we can ship A today safely; B needs another control because failure can duplicate client data.”

Architect follow-up probes

- What quality bar is non-negotiable here?

11. Rapid-fire questions I would still ask

Question	Expected senior answer
PUT vs PATCH?	PUT generally replaces the representation at a resource URI and is expected to be idempotent; PATCH applies a partial modification. Exact semantics depend on the API contract.
401 vs 403?	401 means authentication is missing/invalid; 403 means the identity is understood but not authorized for the operation.
429 handling?	Respect Retry-After where present, back off with jitter, limit concurrency, and do not treat throttling as an ordinary immediate retry.
Why CancellationToken?	Allows request abort, shutdown and timeout budgets to propagate so useless I/O/work can stop cooperatively.
Transient vs permanent failure?	Transient may succeed later without changing the request/data; permanent requires code/config/data/action before retrying is useful.
Database unique constraint or code duplicate check?	Use the database constraint for the invariant. A pre-check alone races under concurrency.
Why correlation IDs?	To connect one logical import/request across API calls, queues, workers, database operations and provider interactions.
Why structured logging?	Fields can be queried/aggregated reliably; parsing free-

	form strings at incident time is fragile.
What is a circuit breaker?	Stops repeated calls to an unhealthy dependency for a period, allowing recovery and protecting resources; it is not a retry mechanism.
What is bulkhead isolation?	Limits resource/concurrency consumption per dependency/workload so one failure domain cannot exhaust the whole service.
What is eventual consistency?	Different parts of the system can temporarily observe different states, but converge through asynchronous processing/reconciliation.
What is optimistic concurrency?	Detect conflicting updates using a version/token and fail/retry instead of locking the record for the whole operation.
What is a poison message?	A message that fails deterministically under current code/data and will not be fixed by immediate retries.
Why not one transaction for a huge import?	Long locks, log growth, rollback cost, contention and poor resumability; use bounded atomic units with checkpoints.
Health check: liveness vs readiness?	Liveness answers whether the process should be restarted; readiness answers whether it can currently receive traffic/work.
Why Docker/Kubernetes here?	Container consistency and orchestration/autoscaling can help workers/services, but they do not solve application-level idempotency, ordering or correctness.

12. Full system-design exercise: model answer

Prompt: “A clinic has 15 years of data and 40 million records. Build a migration that can run while the clinic continues operating, then keep it synchronized afterwards. The central platform must never silently lose or duplicate business records.”

1. Establish identity and consistency boundary

Define clinic/source/entity stable IDs. Capture an import high-water mark or snapshot boundary if the source supports it. Decide how updates occurring during backfill are captured.

2. Extract

Connector reads with keyset/cursor pagination, bounded concurrency, provider rate limiting, explicit timeout/retry classification and durable page/range checkpoints.

3. Stage/envelope

Write each extracted record or batch into durable object storage/staging/queue with ImportId, ClinicId, SourceId, EntityType, source key/version, schema version and checksum where useful.

4. Normalize and validate

Versioned mapping converts source-specific fields into the canonical model. Structural errors and business-rule failures are classified; invalid records are quarantined with reason rather than dropped.

5. Load

Batch/upsert into target using unique source identity and optimistic source-version checks. Use staging tables/bulk techniques for volume. Dependencies are handled by phases or pending-reference resolution.

6. Continuous changes

Webhook/CDC/incremental polling events enter the same idempotent processing path. If backfill and live sync overlap, source version/timestamp rules decide the newest valid state.

7. Completion and reconciliation

Compare source counts/checksums/domain totals where possible; verify all partitions complete and no unresolved quarantine/pending dependencies remain. Mark import complete only after reconciliation criteria pass.

8. Operations

Expose progress/status, queue lag, oldest work age, per-clinic freshness, retries, DLQ/quarantine, reconciliation mismatches and provider throttling. Every record path is traceable by ImportId/correlation ID.

9. Recovery

A crashed worker loses at most the current lease/batch; unfinished work is re-delivered. Idempotent target writes make replay safe. Operators can replay a partition or quarantined record after correction.

10. Rollout

Start with a small clinic/provider cohort, compare outputs, define rollback/disable controls, then increase concurrency gradually based on database/provider saturation metrics.

A good whiteboard shape

Clinic sources -> Connector workers -> durable queue/staging -> normalization/validation -> import workers -> central DB/data platform

Cross-cutting: job/checkpoint store, idempotency/inbox/outbox, DLQ/quarantine, schema/version registry, secrets, metrics/logs/traces, reconciliation tooling.

13. How to answer like a senior instead of a medior

- Start with requirements and failure modes before naming technology.
- Say what must be durable. “I would store the checkpoint” is stronger than “I would retry.”
- State the delivery guarantee: at-least-once + idempotency is a concrete design, not a buzzword.
- Define ordering scope. Avoid saying “messages are ordered” without naming the key/partition.
- For every retry, say which errors are retryable and whether the operation is idempotent.
- For every async design, explain how operators know it is stuck and how they replay it.
- For legacy code, talk about characterization tests, seams, rollout cohorts and rollback instead of rewrite enthusiasm.
- When disagreeing, use evidence, blast radius and business impact. Confidence without evidence is not a senior signal.
- Do not over-architect. If a queue and PostgreSQL solve the actual requirement, do not add Kafka/Kubernetes just because they are in the job ad.

14. Ten questions you should be able to answer flawlessly

1. How do you guarantee an import can resume after a crash without duplicates?
2. How do you handle a POST to a provider when you timed out and do not know whether they processed it?
3. How do you migrate historical data while live updates continue?
4. What is the practical difference between a queue and Kafka for this workload?
5. How do outbox and inbox patterns solve different problems?
6. How do you protect a third-party API from your own autoscaling workers?
7. How do you prove no data was silently lost during an import?

8. How do you safely change mapping logic in a legacy integration?
9. Which metrics tell you that the platform is healthy from the client's perspective?
10. How do you disagree with a client or principal engineer and still move the project forward?

End of interview pack